

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский государственный университет
аэрокосмического приборостроения»

Кафедра №43 «Компьютерных технологий и программной инженерии»

ОТЧЁТ ПО ПРАКТИКЕ
ЗАЩИЩЁН С ОЦЕНКОЙ
РУКОВОДИТЕЛЬ

Ст. преподаватель

должность, уч. степень, звание

С.А. Рогачев

подпись, дата

инициалы, фамилия

ОТЧЁТ ПО ПРАКТИКЕ

вид практики производственная

тип практики преддипломная

на тему индивидуального задания Разработка системы позиционного отслеживания
транспортных средств с обработкой событий в реальном времени

выполнен Быстровым Максимом Денисовичем

фамилия, имя, отчество обучающегося в творительном падеже

по направлению подготовки 09.03.04

код

Программная инженерия

наименование направления

направленности

наименование направления

02

код

Проектирование программных систем

наименование направленности

наименование направленности

Обучающийся группы № 4131з

номер

подпись, дата

М.Д. Быстров

инициалы, фамилия

Санкт-Петербург 2026

ОГЛАВЛЕНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	4
ВВЕДЕНИЕ.....	5
ГЛАВА 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	10
1.1 Описание предметной области.....	10
1.1.1 Современное состояние предметной области	10
1.1.2 Методы и алгоритмы решения задач.....	11
1.1.3 Обзор литературы	14
1.2 Постановка задачи	15
1.2.1 Функциональные требования	15
1.2.2 Нефункциональные требования	18
1.3 Сравнение с аналогами.....	20
1.3.1 Обзор существующих аналогов	20
1.3.2 Сравнительная таблица характеристик	23
1.3.3 Обоснование целесообразности собственной разработки	24
ГЛАВА 2 РАЗРАБОТКА СИСТЕМЫ	25
2.1 Выбор платформы для реализации	25
2.1.1 Операционная система и аппаратное обеспечение	25
2.1.2 Язык программирования и серверные фреймворки.....	26
2.1.3 Системы управления базами данных и кэширования.....	26
2.1.4 Выбор используемых библиотек.....	27
2.1.5 Платформа клиентского веб-интерфейса.....	27
2.1.6 Инфраструктурные решения	28
2.2 Выбор средств разработки	28

2.2.1 Система контроля версий	28
2.2.2 Интегрированные среды разработки	28
2.2.3 Система сборки и управления зависимостями	29
2.2.4 Инструменты тестирования	29
ГЛАВА 3 РЕАЛИЗАЦИЯ И ПОЛУЧЕННЫЕ РЕЗУЛЬТАТЫ.....	30
3.1 Описание разработки программного обеспечения.....	30
3.1.1 Проектирование архитектуры системы.....	30
3.1.2 Описание модулей системы.....	31
3.2 Описание структур данных.....	37
3.2.1 Диаграмма отношений сущностей.....	37
3.2.2 Словарь данных	38
3.3 Описание разработки интерфейса пользователя	44
3.3.1 Интерфейс ведения справочника объектов.....	44
3.3.2 Интерфейс мониторинга транспортных средств.....	46
ЗАКЛЮЧЕНИЕ	50
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	52
Приложение А	55

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

ВВЕДЕНИЕ

Сфера транспортных перевозок является важной частью экономики современного мира и, в частности, Российской Федерации. Посредством различных типов транспорта, таких как железнодорожный транспорт, воздушный транспорт, автомобильный транспорт, водный транспорт в Российской Федерации перевозится не менее 7 миллиардов тонн грузов ежегодно за период с 2000 по 2024 год. Объем пассажирских перевозок за тот же период составляет ежегодно не менее 12,4 миллиардов человек [1].

Большой объем транспортной работы в условиях повсеместной цифровизации ставит перед транспортными предприятиями задачи сбора, хранения, обработки и передачи большого объема информации, связанного с транспортной деятельностью. Эти задачи различного типа не теряют своей актуальности как благодаря растущей конкуренции и закономерному развитию автоматизации с целью повышения эффективности связанных с производственной деятельностью процессов, так и ввиду развития законодательства, предписывающего предприятиям-перевозчикам устанавливать определенное оборудование, отслеживать перевозки с возможностью быстрого реагирования на внештатные ситуации, передавать данные в государственные структуры и социально значимые для населения сервисы.

Например, с 1 сентября 2021 года все транспортные средства категорий М2, М3 и N в обязательном порядке должны быть оборудованы аппаратурой спутниковой навигации, выполняющей передачу данных в Ространснадзор через оператора системы «ЭРА-ГЛОНАСС» [2]. Согласно постановлению Правительства РФ №504 от 14.06.2013 все автомобильные транспортные средства, имеющие разрешенную максимальную массу более 12 тонн, обязаны передавать навигационные данные для автоматического расчета оплаты за пользование дорожной инфраструктурой [3].

Указанные выше факторы привели к появлению и активному развитию на протяжении десятилетий систем отслеживания транспортных средств, призванных решить различные задачи, стоящие перед транспортными предприятиями. Система отслеживания транспортных средств представляет из себя совокупность аппаратного и программного обеспечения, направленную на решение задач, возникающих в работе транспортного предприятия.

Задачи, которые решаются системами отслеживания транспорта, в достаточной степени разнообразны. Каждая система, как правило, решает задачи для предприятий особого профиля, диктующего некоторую специфику перечня задач, которые должны решаться системой, а также допустимые способы решения и ограничения, применяемые при проектировании систем.

В качестве задач, являющихся общими для подобных систем, можно указать следующие: прием сигналов оборудования, формирование, обработка (формирование) телематической информации, хранение телематической информации. Эти задачи являются общими, поскольку обеспечивают доступность данных, используемых для решения других более узких задач.

Также в качестве общих можно указать следующие задачи: зональное отслеживание транспортных средств, пространственно-временной поиск телеметрии, ретрансляция данных телеметрии. Зональное отслеживание позволяет контролировать местоположение ТС, основываясь на заданных географических областях. Пространственно-временной поиск позволяет выполнять поиск телеметрической информации, ограничиваясь рамками определенной географической области и определенного временного интервала. Эти две задачи (проблемы) являются основополагающими для многих методов решения более узких задач автоматизации производственных процессов транспортного предприятия. В свою очередь, ретрансляция данных телеметрии позволяет передавать телематические данные в один или несколько сторонних сервисов, что закрывает потребность передачи данных в предписанные нормативными актами службы.

Таким образом, становится понятен образ базового перечня потребностей транспортного предприятия любой направленности (грузоперевозки, пассажирские перевозки и т.д.), а соответственно, и требований к системам отслеживания транспорта и базовым задачам информатизации, которые должны быть решены в ходе проектирования такой системы.

Рассмотрим подходы, которые могут быть применены в ходе решения ранее перечисленных задач.

Задачи приема, формирования и хранения телеметрической информации сильно взаимосвязаны. С алгоритмической точки зрения эти задачи должны решаться в соответствии с типом использованного оборудования и типом протокола, который используется при приеме данных. Наиболее интересными проблемами, связанными с решением этих задач, являются задачи повышения производительности, масштабируемости и отказоустойчивости сервисов, выполняющих обработку телеметрических данных. Поскольку объем обрабатываемых данных напрямую зависит от количества отслеживаемых объектов, велик риск упереться в предел вычислительных возможностей. Способов решений этих проблем множество: повышение алгоритмической эффективности, наращивание вычислительных мощностей, ограничение нагрузки на уровне конечных устройств, использование распределенных систем и т.д., но наиболее эффективными представляются решения, относящиеся к архитектуре программной системы. С помощью архитектурных решений, таких как использование микросервисного подхода, использование брокеров сообщений, поддержки разделения хранения данных можно создать систему, имеющую высокий потенциал к масштабированию в зависимости от оказываемой на неё нагрузки.

Задача зонального отслеживания транспортных средств сводится к задаче определения вхождения точки на поверхности земного шара (местоположение ТС) в ограниченную географическую область. Эта область задается, как правило, окружностью либо многоугольником (набором точек, образующих вершины

многоугольника). Для определения вхождения могут быть применены следующие методы: перебор зон с проверкой вхождения, применение пространственных индексов в различных СУБД (например, индекс GiST в расширении PostGIS для СУБД Postgresql), использование иерархических пространственных деревьев, использование систем геохеширования (geohash, google s2, uber h3) для создания пространственных индексов в оперативной памяти. Наиболее оптимальным представляется подход с использованием геохеширования, поскольку он обладает рядом преимуществ: имеет легко настраиваемую точность работы (уровень детализации), возможность создания индекса в памяти, занимающего даже на миллионах объектов небольшие объемы ОЗУ, и является предсказуемым по сложности вычисления вхождения, обходя по этому показателю пространственные деревья, которые в сложных ситуациях могут показывать падение производительности и в любом случае требуют проверки геометрического вхождения, определяя лишь набор кандидатов на вхождение. Высокая и предсказуемая производительность метода определения вхождения особенно актуальна в случае, когда выполняется генерация событий вхождения в реальном времени, то есть сразу при приеме данных с оборудования, установленного на транспортных средствах.

Пространственно-временной поиск данных позволяет запрашивать данные, ограничивая запрос по времени поступления информации и по географической области поступления информации. Данные телеметрии хранятся в какой-либо СУБД. На относительно небольших объемах данных и при небольшой нагрузке на базу данных возможно использование только временного индексирования с проверкой вхождения в географическую область на уровне приложения, но с повышением нагрузки такой способ становится неприемлем. Задача может быть решена с помощью комбинации двух индексов – пространственного и по времени – в тех СУБД, которые поддерживают подобный подход. Наиболее оптимальным является применение геохеширования (кодирования) географической позиции телеметрических

данных и составления в результате одного индекса в реляционной СУБД – по результату кодирования и по времени. При этом применяется дискретизация временных и географических показателей с целью увеличения мощности индекса.

Задача ретрансляции сводится к задаче отправки по определенному протоколу ранее полученных данных телеметрии. Сложность алгоритмического проектирования зависит от используемого протокола. Более важной представляется обеспечение отказоустойчивости: обработка недоступности, повтор отправки, контроль допустимой нагрузки. Эти цели достигаются с помощью архитектурного проектирования, отвечающего соответствующим требованиям.

Цель работы – спроектировать и реализовать систему позиционного отслеживания транспортных средств с обработкой событий в реальном времени.

Для достижения цели решаются задачи:

проанализировать предметную область, её современное состояние, обосновать актуальность разработки;

выбрать и описать стек технологий, обосновать выбор;

реализовать программный продукт, описать полученные результаты, провести тестирование разработанного решения.

Объект исследования – процесс обеспечения обработки информации в системах позиционного отслеживания транспортных средств.

Предмет исследования – программный продукт, выполняющий функции позиционного отслеживания транспортных средств с обработкой событий в реальном времени.

Практическая ценность работы заключается в повышении эффективности использования ресурсов, снижении эксплуатационных затрат и увеличении устойчивости систем отслеживания транспорта в условиях высоких нагрузок.

ГЛАВА 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Описание предметной области

Описание предметной области охватывает современное состояние методов решения проблемы с указанием современной литературы, приводится описание возможных способов её решения, их достоинства и недостатки.

1.1.1 Современное состояние предметной области

Современное транспортное предприятие, независимо от его профиля (грузовые, пассажирские, специальные перевозки), функционирует в условиях жесткой конкуренции и возрастающих требований со стороны государства. Законодательство Российской Федерации обязывает оснащать транспортные средства категорий М2, М3 и N аппаратурой спутниковой навигации с передачей данных в систему «ЭРА-ГЛОНАСС» [2], а для автомобилей разрешённой максимальной массой свыше 12 тонн – передавать навигационные данные для взимания платы за использование федеральных дорог [3]. Эти требования формируют устойчивый поток телематической информации, который должен быть принят, обработан и ретранслирован в соответствующие государственные информационные системы.

Помимо нормативных требований, транспортные предприятия решают внутренние производственные задачи, что обуславливает функциональный состав систем отслеживания. Анализ существующих решений и публикаций [4–5] позволяет выделить следующий базовый перечень задач, решаемых такими системами:

1. Приём, обработка и хранение телеметрических данных, поступающих от бортового оборудования различных производителей с использованием различных протоколов.
2. Зональное отслеживание – контроль вхождения транспортного средства в заданные географические области (зоны) и генерация соответствующих событий в режиме, близком к реальному времени.

3. Пространственно-временной поиск телеметрии – выборка данных о местоположении и параметрах движения транспортных средств с одновременными ограничениями по географическому региону и временному интервалу.
4. Ретрансляция телеметрических данных во внешние сервисы, включая государственные системы (ЭРА-ГЛОНАСС, «Платон»), с обеспечением гарантированной доставки и устойчивости к сбоям.

Каждая из перечисленных задач обладает собственной спецификой, порождающей требования к алгоритмическим и архитектурным решениям.

1.1.2 Методы и алгоритмы решения задач

1. Приём, обработка и хранение телеметрии.

Поток навигационных данных от тысяч транспортных средств носит асинхронный и высокоинтенсивный характер. Традиционный монолитный подход к построению подсистемы приёма данных плохо масштабируется и не обеспечивает требуемой отказоустойчивости [6]. В современной литературе в качестве стандарта для решения подобных задач рассматриваются архитектуры, основанные на микросервисах и брокерах сообщений (Apache Kafka, RabbitMQ) и разделении хранилищ на оперативные (базы данных в ОЗУ, кэши) и аналитические (реляционные СУБД, колоночные хранилища) [7]. Такое разделение позволяет независимо масштабировать компоненты приёма, первичной обработки и долговременного хранения, а также изолировать сбои, предотвращая отказы.

2. Зональное отслеживание.

Задача определения принадлежности точки с географическими координатами заданной области является классической задачей вычислительной геометрии. Для многоугольных зон применяются алгоритмы, такие как метод трассировки луча [8]. Однако при необходимости обработки миллионов сообщений и тысяч зон прямой перебор зон с полной геометрической проверкой

неприемлем из-за вычислительной сложности $O(K \cdot N)$, где K – число зон, N – число точек.

Для снижения сложности используют методы пространственного индексирования:

- Пространственные индексы в СУБД (например, GiST в PostGIS). Такой подход эффективен для пакетной и аналитической обработки, но при использовании потоковой архитектуры вызывают задержки, связанные с обращениями к дисковой подсистеме [9].
- Иерархические пространственные деревья. Обеспечивают логарифмическую сложность поиска кандидатов, однако в худших случаях (неравномерное распределение, вытянутые полигоны) могут требовать проверки большого числа кандидатов и сложны в балансировке [10].
- Системы геохеширования и сферической дискретизации, самыми популярными представителями которых можно назвать geohash, Google S2, Uber H3. Эти методы набирают популярность благодаря возможности представить географическую область в виде набора дискретных ячеек и выполнять проверку вхождения точечного объекта за логарифмическое количество операций бинарного поиска по предварительно рассчитанному интервальному индексу, хранимому в оперативной памяти. В частности, библиотека Google S2 позволяет строить компактные индексы, сохраняющие предсказуемую производительность при любом количестве и форме зон, что критически важно для обработки событий в реальном времени [11, 12].

Таким образом, для систем с жёсткими временными ограничениями наиболее перспективным представляется использование пространственных индексов на основе системы геохеширования S2, обеспечивающих высокую скорость проверки вхождения ценой предварительного разложения зон на наборы ячеек заданного уровня.

3. Пространственно-временной поиск.

Запросы, объединяющие пространственные и временные фильтры, являются типовым инструментом диспетчерского анализа. Классический подход заключается в создании составных индексов в реляционных СУБД (например, B-tree индекс по столбцам `sector_id`, `timestamp`), где идентификатор сектора получают заранее определённой пространственной дискретизацией [11]. Альтернативой выступают специализированные time-series базы данных с поддержкой геопространственных типов (TimescaleDB + PostGIS), которые позволяют эффективно обрабатывать подобные запросы за счёт партиционирования по времени и использования пространственных индексов внутри партиций [9]. Также возможен вариант с использованием двух не связанных между собой индексов в тех СУБД, которые поддерживают построение плана по разным индексам для одного запроса [13]. В системах, ориентированных на высокую нагрузку, применяется предварительная дискретизация как временных, так и пространственных показателей с построением единого индекса, что максимизирует плотность упаковки данных и уменьшает количество операций ввода-вывода. Наиболее производительным и гибким решением представляется составной индекс в СУБД по времени и идентификатору сектора, поскольку мощность такого индекса легко контролировать изменением уровня геохеширования, нет необходимости партиционирования, которое уменьшает эффективность запросов, не задействующих ключ партиционирования, а также не зависит от планировщика запросов так же сильно, как подход с двумя отдельными индексами.

4. Ретрансляция данных.

Алгоритмическая сложность задачи ретрансляции зависит от целевого протокола (EGTS, HTTP, MQTT) и требований к гарантии доставки. Основная инженерная проблема лежит в области обеспечения отказоустойчивости: обработка временной недоступности внешнего сервиса, повтор отправки с экспоненциальной отсрочкой, контроль входящего потока. Эти задачи решаются

архитектурными средствами – введением исходящих очередей с механизмом повторной обработки неудачных сообщений и стратегией очереди недоставленных писем (DLQ) для окончательно недоставленных данных [7].

1.1.3 Обзор литературы

Вопросам построения систем мониторинга и управления транспортом посвящён ряд научных работ и отраслевых публикаций. В статье А.Н. Зиарманда и В.И. Хаханова «Модели и методы мониторинга и управления транспортом» [4] рассмотрены основные направления развития киберфизического транспортного мониторинга, включая средства точного позиционирования, облачные сервисы и системы мониторинга в реальном времени. Монография П.И. Смирнова, М.Ю. Карелиной и Б.С. Субботина «Применение систем транспортной телематики для повышения эффективности эксплуатации техники» (2023) [14] представляет математические модели и концепции управления бизнес-процессами предприятий на основе данных спутникового мониторинга.

Проблематике масштабируемости серверной части систем отслеживания транспорта посвящена статья [5], в которой рассматриваются особенности существующих решений и предлагается подход к построению единой платформы на основе сервис-ориентированной архитектуры. Технологии потоковой обработки данных — Apache Kafka и RabbitMQ — рассмотрены в обзоре [6], где показаны возможности построения отказоустойчивых конвейеров обработки потоковых данных.

Методам геопространственного индексирования, имеющим ключевое значение для задачи зонального отслеживания, посвящена статья [15]. В работе выполнен сравнительный анализ трёх наиболее распространённых в коммерческих приложениях систем геокодирования, рассмотрены алгоритмические особенности каждой из них, включая форму ячейки, число уровней иерархии и вычислительную сложность операций.

Задача пространственно-временного поиска телеметрических данных исследована в работах М. Gorawski и А. Dyga. В статье «Indexing of Spatio-

Temporal Telemetric Data based on Distributed Mobile Bucket Index» [16] предложена структура данных, отвечающая на пространственно-временные диапазонные запросы к телеметрическим данным, с возможностью адаптивной настройки под изменяющееся распределение данных.

Протокол EGTS (Era Glonass Telematics Standard) действует в соответствии с актуальным стандартом ГОСТ 33465-2015 [17]. Протокол является обязательным для использования при передаче данных в систему «ЭРА-ГЛОНАСС».

Проведённый обзор подтверждает актуальность задачи создания системы позиционного отслеживания транспортных средств с обработкой событий в реальном времени на основе современных подходов: микросервисной архитектуры с потоковой обработкой данных, пространственных индексов в памяти на базе Google S2 и дискретизированных индексов для пространственно-временного поиска. Данные решения позволяют добиться высокой производительности, предсказуемого поведения под нагрузкой и отказоустойчивости, что является ключевыми критериями качества для транспортных телематических платформ.

1.2 Постановка задачи

На основании проведённого анализа предметной области, существующих систем-аналогов и современных технологических подходов сформулированы цель и требования к разрабатываемой системе позиционного отслеживания транспортных средств с обработкой событий в реальном времени.

Цель разработки – создание высокопроизводительной, масштабируемой и отказоустойчивой программной платформы, обеспечивающей приём, обработку, хранение телеметрической информации, зональное отслеживание транспортных средств в режиме реального времени, пространственно-временной поиск телеметрии, а также ретрансляцию данных во внешние системы.

1.2.1 Функциональные требования

Система должна обеспечивать выполнение следующих основных функций:

1. Приём телеметрических данных:

- приём навигационных и диагностических пакетов от бортового оборудования различных производителей;
- поддержка протокола EGTS как обязательного стандарта взаимодействия с государственными системами и обязательным бортовым оборудованием;
- потоковая обработка входящих сообщений с гарантией сохранения порядка в рамках одного транспортного средства;
- подтверждение приема данных только после подтверждения их отправки во внутренний брокер сообщений.

2. Первичная обработка и хранение телеметрии:

- анализ показателей телеметрии; формирование событий, связанных с телеметрическими показателями;
- сохранение данных в СУБД для дальнейших запросов.

3. Зональное отслеживание в реальном времени:

- ведение реестра геозон, описываемых многоугольниками и окружностями;
- автоматическое определение факта вхождения/выхода транспортного средства в/из контролируемой зоны в момент поступления телеметрического пакета;
- генерация событий (вход, выход, превышение скорости в зоне и др.) и их доставка заинтересованным подсистемам;
- использование пространственного индекса в оперативной памяти на базе геохеширования Google S2 для обеспечения отслеживания в реальном времени при количестве зон 50 000 в одном модуле отслеживания при условии соответствия системы системным

требованиям. Количество хранимых в справочнике геозон должно быть ограничено 1 000 000 зонами, которое на практике в большинстве случаев недостижимо. Возможность обработки большего числа зон, чем 50 000 в реальном времени должна быть обеспечена возможностью горизонтального масштабирования и наращивания ресурсов системы.

4. Пространственно-временной поиск телеметрии:

- выборка телеметрических записей по комбинированному фильтру: географический регион (прямоугольник) и временной интервал;
- использование дискретизированных пространственно-временных индексов (составной индекс по идентификатору ячейки (геохеш) и времени) для эффективного выполнения запросов;
- предоставление пользовательского интерфейса для отображения треков за запрошенный период на карте, вывод списка транспортных средств, находившихся за указанный временной интервал в запрошенной географической области.

5. Ретрансляция данных:

- возможность настройки передачи обработанных телеметрических пакетов в одну или несколько внешних систем по различным протоколам (EGTS, HTTP/JSON);
- настраиваемые правила маршрутизации: тип данных, идентификатор транспортного средства;
- механизм гарантированной доставки: буферизация и повторные попытки с экспоненциальной отсрочкой, выделенная очередь недоставленных сообщений.

6. Пользовательский интерфейс:

- веб-интерфейс для администраторов и диспетчеров, обеспечивающий визуализацию текущего положения транспортных

средств на карте, историю треков, конфигурирование геозон, правил ретрансляции и просмотр событий;

- REST API для интеграции с внешними информационными системами предприятия (учётные системы, системы документооборота, мобильные приложения).

7. Требования к данным

Входные данные: телеметрические пакеты, содержащие идентификатор устройства, географические координаты, скорость, направление, высоту, показания датчиков, временную метку; пакеты поступают от бортовых навигационных терминалов.

Выходные данные:

- события телеметрического контроля (тип события, объект, время, координаты, метаданные)
- события зонального контроля (тип события, объект, зона, время);
- результаты пространственно-временных запросов (набор телеметрических записей);
- ретранслированные пакеты в формате, требуемом принимающей стороной.

Хранение: долговременное хранение всех необработанных и обработанных телеметрических записей в реляционной СУБД; хранение конфигураций (геозоны, правила) в реляционной СУБД, кратковременное хранение данных в персистентном брокере сообщений (Apache Kafka).

1.2.2 Нефункциональные требования

1. Производительность:

- система должна обрабатывать поток не менее 2 000 сообщений в секунду на рекомендуемом аппаратном обеспечении (Приложение 1);

- задержка между поступлением пакета в приемную службу и генерацией события входа/выхода из зоны не должна превышать 500 мс в 95% случаев при нагрузке 2 000 сообщений в секунду.

2. Масштабируемость:

- архитектура должна позволять горизонтальное масштабирование компонентов приёма, обработки, хранения и ретрансляции данных;
- должен быть использован микросервисный подход и применено асинхронное взаимодействие через брокер сообщений для независимого расширения узких мест.

3. Отказоустойчивость:

- система должна сохранять работоспособность при временной недоступности внешних сервисов (очереди повторной отправки);
- предусмотреть аппаратное разделение критичных компонентов для устранения единой точки отказа и уменьшении вероятности полной потери работоспособности;
- автоматическое восстановление после сбоев без потери уже принятых данных.

4. Безопасность:

- аутентификация и авторизация пользователей веб-интерфейса и API;
- межсервисное взаимодействие по защищённым каналам (SSL/TLS);
- журналирование действий пользователей и административных операций.

5. Технологические требования

- серверные приложения должны быть реализованы с использованием языка программирования Kotlin;
- для разработки клиентского веб-интерфейса должен быть использован компонентно-ориентированный фронтенд-фреймворк Vue, язык программирования – TypeScript/JavaScript;

- в качестве брокера сообщений и потоковой шины используется Apache Kafka и RabbitMQ;
- основная СУБД – PostgreSQL, для кэширования применяется Redis;
- пространственное индексирование зон выполняется с помощью геохеширования Google S2;
- развёртывание выполняется в контейнерах Docker для упрощения эксплуатации и масштабирования.

6. Сопровождение и документация:

- код должен сопровождаться технической документацией и руководством по развёртыванию;
- предусмотрены средства мониторинга работоспособности (метрики поступления, задержек, ошибок ретрансляции).

1.3 Сравнение с аналогами

Для оценки конкурентоспособности разрабатываемой системы позиционного отслеживания транспортных средств с обработкой событий в реальном времени был проведён сравнительный анализ наиболее распространённых на рынке решений. Основное внимание уделялось функциональным возможностям, производительности геозон, поддержке протокола EGTS и государственных информационных систем, масштабируемости и отказоустойчивости.

1.3.1 Обзор существующих аналогов

1. Traccar

Открытая платформа спутникового мониторинга (лицензия Apache 2.0), поддерживающая широкий спектр протоколов телематического оборудования. Функциональность включает отображение текущего положения, историю треков, отчёты и базовый геозонинг.

Преимущества: бесплатное распространение, большое сообщество, простой REST API, есть поддержка протокола EGTS и множества других

протоколов. Присутствует широкая поддержка ретрансляции данных: как «сырых», так и предварительно обработанных.

Недостатки: геозональное отслеживание, хоть и не приводит к одиночным запросам при каждой проверке, используя кеширование, тем не менее, не использует пространственный индекс, ввиду чего при достижении большого количества геозон скорость обработки будет сильно падать [18]. Также отсутствует встроенная возможность пространственно-временного поиска телеметрических данных.

Пространственно-временной поиск отсутствует.

2. Wialon

Проприетарная платформа, один из лидеров рынка. Предоставляет развитый интерфейс, сотни типов отчётов, продвинутое зональное отслеживание и интеграцию с датчиками. Поддерживает ретрансляцию данных по различным протоколам, включая EGTS. Имеет локальную версию.

Преимущества: высокая надёжность (заявлено время доступности 99,98 %), регулярные обновления, поддержка большинства оборудования, мощная аналитика.

Недостатки: высокая стоимость лицензий, особенно для крупных автопарков; закрытый исходный код ограничивает возможности кастомизации. С 1 января 2026 года все операции на территории РФ прекращены.

Количество геозон и других элементов контроля ограничено количеством 31744, чего может быть недостаточно для крупного транспортного предприятия [19]. Из-за закрытости платформы и ограничения количества геозон невозможно достоверно выяснить, используются ли алгоритмы пространственной индексации. При заявленном трафике 944 точек в секунду и ограничения количества геозон гарантированная вычислительная сложность может быть значительно превышена с использованием подходов, предлагаемых в данной работе.

Пространственно-временной поиск отсутствует.

3. Navixy

Коммерческая платформа. Обладает широкими возможностями интеграций с различным оборудованием, ретрансляцией данных. Имеет локальную версию. В РФ продается в локализованном виде под названием «ГМ Телематика».

Преимущества: гибкая система отчётов, удобный конструктор дашбордов, поддержка EGTS, есть ретрансляция.

Недостатки: для получения конкурентоспособной производительности геозон требуется горизонтальное масштабирование, которое в локальной версии ограничено; зависимость от вендора.

Пространственно-временной поиск отсутствует.

4. 1С:Центр спутникового мониторинга ГЛОНАСС/GPS

Российское решение, интегрированное с платформой 1С:Предприятие. Позволяет организовать мониторинг транспорта в единой информационной среде предприятия, поддерживает работу с оборудованием, передающим данные в систему ЭРА-ГЛОНАСС.

Преимущества: тесная интеграция с учётными системами 1С, соответствие требованиям российского законодательства, поддержка ретрансляции. Есть события отслеживания в реальном времени.

Недостатки: временная сложность обработки зонального отслеживания линейная, пространственная индексация отсутствует. Уже на количестве около 200 ТС обработка входов/выходов в геозоны может занимать значительное количество времени. Пространственно-временной поиск отсутствует. Технических ограничений на количество ТС и зон нет, но лицензия приобретается на определенное количество объектов мониторинга, ввиду чего обслуживание крупного автопарка с помощью 1С может стать невыгодным, кроме того, на большом количестве объектов мониторинга скорость работы и количество затрат на вычислительные ресурсы могут стать неудовлетворительными.

5. Fort Monitor

Отечественная система мониторинга транспорта, сертифицированная для использования в государственных структурах. Поддерживает протокол EGTS и ретрансляцию в Ространснадзор.

Преимущества: полное соответствие российским нормативным требованиям, развитая картографическая подсистема, собственное серверное решение.

Недостатки: закрытость архитектуры; гарантированная скорость работы системы (в среднем 50 сообщений в секунду) с учетом задекларированных системных требований не позволяет сделать вывод о наличии высокоэффективной пространственной обработки [20]. Пространственно-временной поиск отсутствует. Возможность горизонтального масштабирования не описана.

1.3.2 Сравнительная таблица характеристик

В таблице ниже приведено сравнение ключевых функциональных и нефункциональных характеристик рассмотренных аналогов с целевыми показателями разрабатываемой системы.

Таблица 1 Сравнение аналогов

Характеристика	Разрабатываемая система (целевая архитектура)	Traccar	Wialon	Navixy	1С:ЦСМ	Fort Monitor
Поддержка EGTS	Да	Да	Да	Да	Да	Да
Производительность зонального отслеживания	Очень высокая	Низкая	Высокая	Высокая	Низкая	Средняя

Характеристика	Разрабатываемая система (целевая архитектура)	Traccar	Wialon	Navixy	1С:ЦСМ	Fort Monitor
Пространственно-временной поиск	Да	Нет	Нет	Нет	Нет	Нет
Масштабируемость	Горизонтальная	Горизонтальная	Вертикальная	Горизонтальная /Вертикальная	Вертикальная	Вертикальная
Открытость кода	-	Открытый	Закрытый	Закрытый	Закрытый	Закрытый
Лицензионная стоимость	-	Бесплатно	Высокая	Средняя/высокая	Средняя	Средняя
Доступность в РФ	-	Да	Нет	Локализация	Да	Да

1.3.3 Обоснование целесообразности собственной разработки

Анализ показывает, что ни одно из существующих решений не обеспечивает одновременно:

- высокой производительности зонального отслеживания в режиме реального времени;
- пространственно-временного поиска телеметрических данных;
- возможности горизонтального масштабирования.

Таким образом, разработка собственной системы позиционного отслеживания с заявленными характеристиками является обоснованной. Предлагаемое решение позволит транспортным предприятиям повысить эффективность управления парком, сократить расходы на инфраструктуру,

получить новые аналитические возможности, получить продукт, который может быть использован даже при значительном увеличении количества объектов мониторинга.

ГЛАВА 2 РАЗРАБОТКА СИСТЕМЫ

2.1 Выбор платформы для реализации

Выбор технологического стека является важным этапом проектирования системы позиционного отслеживания транспортных средств, поскольку он напрямую определяет достижимость предъявленных к системе нефункциональных требований: производительности, масштабируемости, отказоустойчивости и безопасности.

2.1.1 Операционная система и аппаратное обеспечение

В качестве целевой операционной системы для серверной части выбрана ОС семейства Linux Ubuntu 24.04 LTS. Этот выбор продиктован следующими факторами:

- наилучшей поддержкой контейнеризации и инструментов оркестрации (Docker, Kubernetes);
- популярностью дистрибутивов Ubuntu, развитое сообщество;
- преимуществами выпуска LTS (длительный период выпуска системных обновлений и патчей безопасности);
- бесплатность операционной системы для любого вида использования.

Аппаратная конфигурация целевого сервера определяется необходимостью обслуживать входящий поток не менее 2000 телеметрических сообщений в секунду. Исходя из практики развёртывания аналогичных систем, достаточными являются:

- CPU: 8 физических ядер с тактовой частотой не ниже 3.0 ГГц;
- оперативная память: 16 ГБ (для размещения пространственного индекса, кэша Redis и рабочих процессов сервисов);

- дисковая подсистема: SSD-накопитель объёмом не менее 500 ГБ (под оперативное хранение очередей Kafka), а также накопитель объёмом от 1 ТБ для непосредственно базы данных PostgreSQL (размер требуемого хранилища зависит от периода, за который должны храниться данные).

Клиентские рабочие места не предъявляют жёстких требований: достаточно, чтобы на рабочем ПК работал любой из современных браузеров (Google Chrome, Mozilla Firefox, Microsoft Edge и др.) и производительности было достаточно для приемлемой задержки отклика с точки зрения пользователя.

2.1.2 Язык программирования и серверные фреймворки

Язык программирования Kotlin должен быть использован для разработки серверных модулей, поскольку одна из активно развиваемых библиотек для работы с геохешированием S2 предназначена для работы в JVM. В качестве серверного фреймворка будет использован Spring Boot версии не ниже 3.5. Spring Boot, веб-фреймворк с открытым исходным кодом, является лидером среди фреймворков для построения серверных приложений для JVM, являясь надежным и удобным решением, развиваясь с участием сообщества разработчиков. Также язык Kotlin обладает высокой производительностью несмотря на то, что работает в виртуальной машине, благодаря оптимизациям и JIT-компиляции.

2.1.3 Системы управления базами данных и кэширования

В качестве основной СУБД выбрана PostgreSQL 16. Определяющими факторами послужили:

- популярность СУБД, её активное развитие и богатая документация;
- высокий уровень производительности;
- наличие расширений для обработки географических данных (PostGIS), что может быть полезно при будущем развитии системы.

Для кэширования и организации очередей сообщений выбраны два продукта:

- Redis для кэширования и межсервисного взаимодействия. Его использование подтверждено успешным опытом в системах корпоративного телевидения [9], где требовалось быстрое обновление данных без обращения к дисковой СУБД.
- Apache Kafka в качестве распределённого брокера сообщений, образующего ядро потоковой шины. Kafka обладает возможностью горизонтального масштабирования и репликации, что позволяет организовать надежную и производительную потоковую обработку данных.

2.1.4 Выбор используемых библиотек

Наиболее подходящим является метод геохеширования на основе библиотеки S2 Geometry Library For Java, который обеспечивает:

- представление географических областей в виде наборов дискретных ячеек (интервалов ячеек) что позволяет заменить проверку геометрического вхождения на быстрый бинарный поиск по предварительно рассчитанному интервальному индексу, хранимому в оперативной памяти;
- логарифмическую сложность проверки, предсказуемое время выполнения даже для сложных многоугольных зон;
- возможность гибко регулировать точность представления, управляя потреблением памяти.

Таким образом, будет использована библиотека S2 Geometry Library For Java.

2.1.5 Платформа клиентского веб-интерфейса

Клиентская часть должна быть реализована с применением компонентно-ориентированного фронтенд-фреймворка Vue.js на языке TypeScript/JavaScript. Основанием для такого выбора является:

высокая скорость разработки динамических интерфейсов благодаря реактивной модели данных;

компонентный подход, реализуемый фронтенд-фреймворком Vue и позволяющий переиспользовать код и разметку в разных частях приложения;

наличие экосистемы готовых компонентов для отображения карт, построения таблиц и графиков, что особенно ценно для диспетчерских панелей;

возможность строгой типизации на TypeScript, снижающей количество ошибок на этапе разработки.

2.1.6 Инфраструктурные решения

Упаковка всех серверных компонентов в Docker-контейнеры обеспечивает идентичность окружения на этапах разработки, тестирования и промышленной эксплуатации, упрощает развёртывание и горизонтальное масштабирование. Для управления растущим количеством контейнеров будет применен Docker Compose. Организация CI/CD пайплайна с использованием GitHub Actions или GitLab CI позволит автоматизировать сборку образов и их публикацию в реестр.

Таким образом, предложенный стек удовлетворяет функциональным и нефункциональным требованиям, сформулированным в разделе 1.2, и создаёт необходимый фундамент для реализации высокопроизводительной, горизонтально масштабируемой системы спутникового мониторинга.

2.2 Выбор средств разработки

2.2.1 Система контроля версий

В качестве системы контроля версий применяется Git. Для хранения исходного кода и организации командной работы выбрана платформа GitHub. Сервис GitHub имеет функционал GitHub Actions, который позволяет автоматизировать тестирование, сборку и развёртывание на основе данных репозитория.

2.2.2 Интегрированные среды разработки

Для серверной части системы, реализуемой на языке Kotlin, основной средой разработки служит OpenIde, являющаяся российской средой разработки,

удовлетворяющей российскому законодательству и основанной на IDE IntelliJ IDEA компании JetBrains, которая также является автором языка Kotlin, благодаря чему обеспечивается широкая поддержка языка программирования, применяемых фреймворков и разнообразных расширений.

Для разработки клиентского веб-интерфейса на Vue.js и TypeScript применяется редактор Visual Studio Code. Его популярность в сообществе фронтенд-разработчиков и богатый выбор расширений (в том числе и от разработчиков Vue) делают его предпочтительным инструментом для быстрой и качественной работы с реактивными интерфейсами.

2.2.3 Система сборки и управления зависимостями

В серверных микросервисах используется Gradle с Kotlin DSL. Для выполнения сборки и публикации docker-образа использован плагин Gradle Google Jib. Для фронтенд-части в качестве сборщика применяется Vite. Обе системы сборки являются наиболее распространенными для своего стека технологий.

2.2.4 Инструменты тестирования

Обеспечение высокого качества разрабатываемой системы требует многоуровневого тестирования. Модульное тестирование серверных компонентов выполняется с использованием JUnit. Для проверки REST API применяется Bruno, что даёт возможность создавать и выполнять коллекции запросов, проверяя корректность ответов.

ГЛАВА 3 РЕАЛИЗАЦИЯ И ПОЛУЧЕННЫЕ РЕЗУЛЬТАТЫ

3.1 Описание разработки программного обеспечения

3.1.1 Проектирование архитектуры системы

Проектирование архитектуры – процесс планирования внутреннего устройства системы: определение количества и типов подсистем, связей между ними. Для наглядного представления данного этапа проектирования была выбрана нотация диаграммы компонентов, которая представлена на рисунке 1.

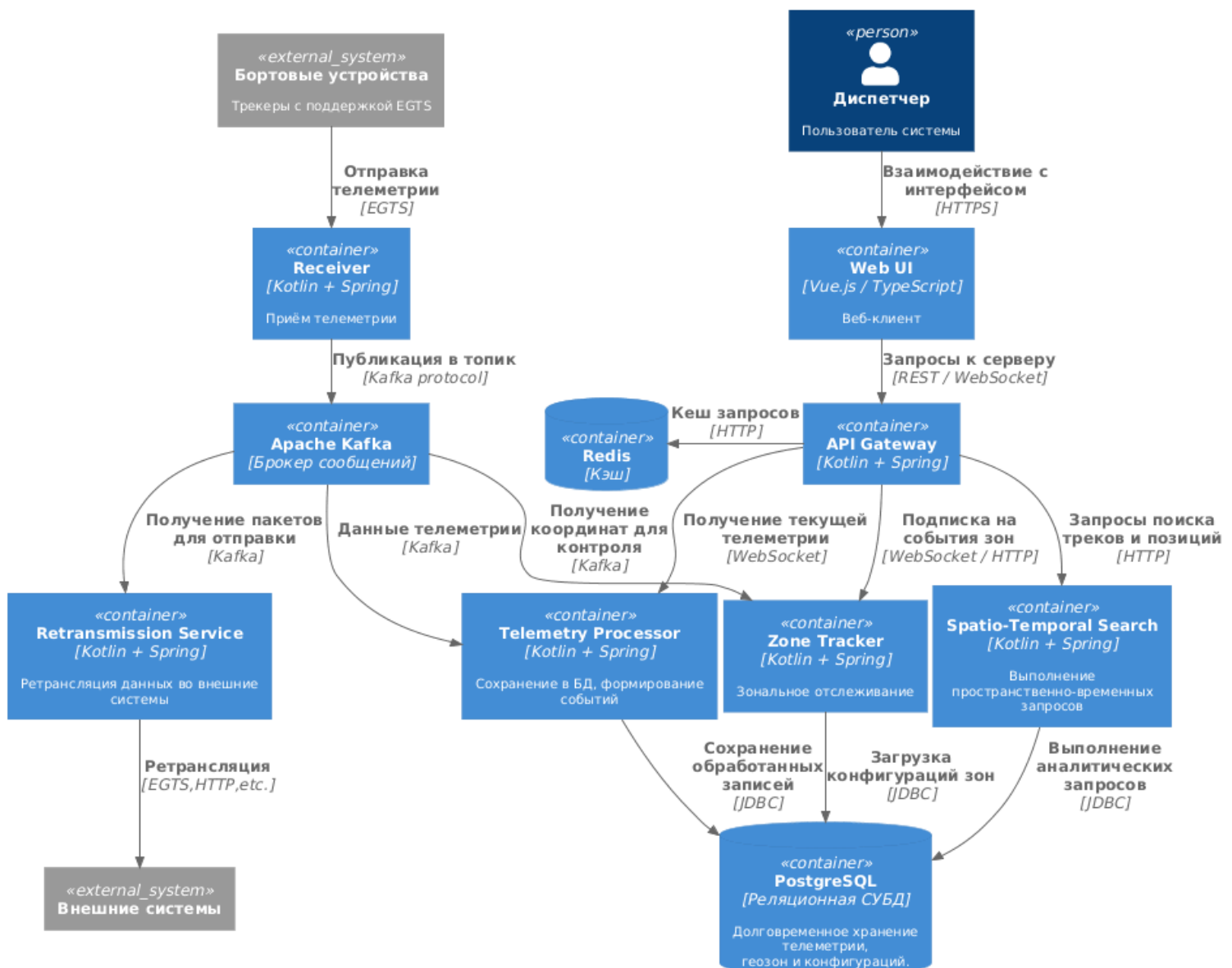


Рисунок 1 Архитектура системы

Архитектура системы на рисунке 1 представлена в минимальном исполнении, когда каждый из модулей представлен в единственном экземпляре. При масштабировании системы количество модулей может быть увеличено: могут быть добавлены дополнительные приемники, передатчики и обработчики телеметрии, генераторы событий.

3.1.2 Описание модулей системы

Далее будет приведено описание каждой из подсистем разрабатываемой системы, указано их назначение, значимые подробности об их внутреннем устройстве, связи с другими модулями, протоколы обмена, состав данных.

1. Клиентское веб-приложение (Web UI)

Клиентское веб-приложение, предоставляющее пользовательский интерфейс для мониторинга транспортных средств, просмотра треков, событий, ведения справочника геозон и других настроек.

Модуль осуществляет взаимодействия, представленные в таблице 2.

Таблица 2 Связи клиентского модуля

Целевой модуль	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
Шлюз API	Отправка запросов и получение ответов в ходе любых операций пользователя	HTTP, WebSocket	HTTP: JSON с параметрами запросов, ответы в JSON (списки ТС, треки, события). WebSocket: сообщения с уведомлениями о событиях.

2. Шлюз API (API Gateway)

Единая точка входа для клиентского приложения. Маршрутизирует запросы к соответствующим модулям.

Таблица 3 Связи шлюза API

Целевой модуль	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
Обработчик телеметрии	Получение актуальной телеметрии и событий	WebSocket	Запрос: параметры фильтрации (список ТС) Ответ: поток актуальных телеметрических данных и событий
Модуль зонального отслеживания	Получение событий зон	WebSocket	Запрос: параметры фильтрации (список ТС) Ответ: поток событий геозон
Модуль пространственно-временного поиска	Поиск треков, ТС в заданном регионе за период, поиск телеметрии по ТС	HTTP	Запрос: фильтры по ТС, зона поиска. Ответ: список ТС. Запрос: ТС, временной интервал Ответ: массив телеметрических данных
Redis	Запрос закешированных данных	Redis	Картинки, конфигурация, метаданные объектов системы

3. Приемная служба (Receiver)

Приём телеметрических пакетов от бортовых устройств по протоколу EGTS. Обеспечивает аутентификацию оборудования и разбор входящих данных, отправку данных в брокер сообщений.

Таблица 4 Связи приемной службы

Целевой контейнер	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
Брокер сообщений	Передача принятых телеметрических данных	Kafka protocol	Разобранные телеметрические данные

4. Брокер сообщений (Apache Kafka)

Центральная асинхронная шина сообщений, развязывающая источники и потребителей данных. Обеспечивает гарантированную доставку, масштабируемость и отказоустойчивость.

Основным понятием обмена, используемом в Kafka, является топик (topic). В топике хранятся сообщения - бинарные данные ключ-значение. Каждый из топиков может быть распределен между несколькими серверами кластера. Сообщения в топике хранятся определенное ограниченное время, после которого удаляются (также возможно ограничение по размеру топика), до удаления возможна перечитка данных. Топик делится на партиции, распределение по которым происходит с помощью хеширования ключа сообщения. Kafka поддерживает практически неограниченное расширение кластера – до сотен серверов в одном кластере, что делает способность к горизонтальному сообщению очень высокой.

Модули, работающие с брокером сообщений, могут выступать в качестве производителей либо потребителей сообщений. При доставке сообщений в большинстве случаев используется модель доставки по подписке, когда между потребителем и брокером постоянно сохраняется соединение, и в момент поступления события брокер инициирует отправку сообщения потребителю. Такая модель позволяет серверу не знать адресов потребителей, но при этом с минимальной задержкой обеспечивать доставку сообщений по назначению.

5. Обработчик телеметрии (Telemetry Processor)

Потребляет поток телеметрических данных из брокера сообщений, выполняет формирование событий, сохраняет обработанные записи в базу данных. Формирует для каждого телеметрического сообщения S2-ключ заданного уровня для использования в пространственно-временном поиске.

Обработчик формирует события, исходя из показателей телеметрии, при нарушениях установленных ограничений. Описание структуры показателей телеметрии, возможных видов задания телеметрии будет приведено далее при описании структур данных.

Таблица 5 Связи обработчика телеметрии

Целевой модуль	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
База данных	Сохранение нормализованных телеметрических записей для последующего анализа и поиска.	JDBC	Основные данные (данные записи); дополнительные данные (данные подзаписей)

6. Модуль зонального отслеживания

Реализует зональное отслеживание в реальном времени. Потребляет телеметрические данные из общего потока Kafka, с помощью пространственного индекса на базе Google S2 определяет факт вхождения/выхода ТС в геозоны и генерирует соответствующие события. Описание алгоритма геохеширования зон, формирования индекса в памяти и использования его при работе приведено в Приложении 1.

Таблица 6 Связи модуля зонального отслеживания

Целевой контейнер	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
PostgreSQL	Загрузка актуального списка геозон и их параметров Сохранение событий посещений зон	JDBC	Данные геозон: тип, объекты описания (точки) Сохранение записи о входе/выходе из геозоны
Redis	Сохранение/загрузка зон, в которых ТС находится в данный момент	Redis	Ключ – ТС, значение – список текущих зон

7. Модуль пространственно-временного поиска (Spatio-Temporal Search)

Модуль выполняет поиск телеметрических данных по географическому интервалу и временному интервалу; по ТС и временному интервалу.

Таблица 7 Связи модуля пространственно-временного поиска

Целевой контейнер	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
PostgreSQL	Выполнение пространственно-временных SQL-запросов с использованием составного индекса Поиск телеметрических данных для ТС	JDBC	Запрос: временной интервал, пространственный интервал Ответ: ТС, находившиеся в заданном временном промежутке в заданный пространственный интервал Запрос: ТС, временной интервал Ответ: телеметрические данные

8. Передатчик телематики (Retransmission Service)

Обеспечивает доставку телеметрических данных во внешние системы в соответствии с требуемыми протоколами. Реализует механизм повторных попыток и очередь недоставленных сообщений (DLQ).

Таблица 8 Связи передатчика телематики

Целевой контейнер	Назначение обращения	Протокол	Состав передаваемых/получаемых данных
Внешние системы (ЭРА-ГЛОНАСС, Платон)	Отправка навигационных пакетов в установленном формате.	EGTS, HTTP/JSON	Структура пакетов, определённая протоколом EGTS (ГОСТ Р 54619-2011) или JSON-структура, специфичная для системы-получателя.

9. Redis

Высокопроизводительное хранилище в оперативной памяти для кэширования оперативной информации, обеспечивающее снижение издержек на работу базы данных при работе системы. Используется шлюзом API для уменьшения количества запросов к другим сервисам, и как следствие, для повышения производительности системы.

10. PostgreSQL

Основное реляционное хранилище для долговременного хранения нормализованных телеметрических данных, конфигураций геозон, правил ретрансляции, настроек событий, событий и прочих данных. Выбрана версия PostgreSQL 17 – последняя стабильная версия на момент проектирования системы.

Обслуживает SQL-запросы от других модулей. Состав хранимых данных будет описан при описании структур данных.

Встроенные возможности горизонтального масштабирования PostgreSQL ограничены, применяются дополнительные решения для шардирования баз данных.

3.2 Описание структур данных

Хранение данных в системе обеспечивается посредством двух модулей: базы данных и брокером сообщений. Поскольку данные, передаваемые в брокере сообщений, также представлены и в базе данных, достаточно будет описания структур данных в БД для полного описания структур всех сущностей, существующих в системе.

3.2.1 Диаграмма отношений сущностей

Для описания БД на рисунке 2 представлена диаграмма отношений сущностей (ER-diagram).

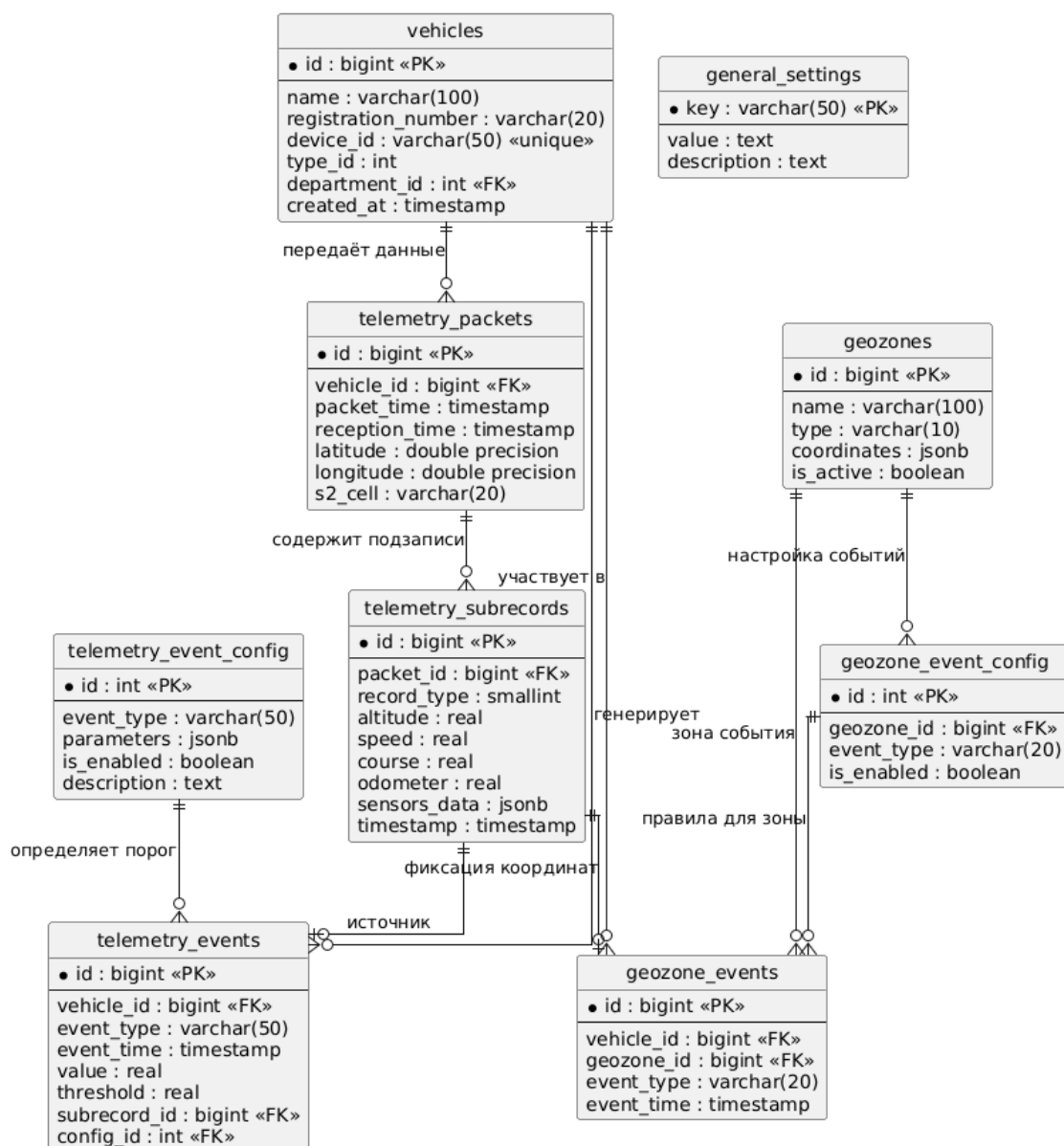


Рисунок 2 Схема отношений сущностей

3.2.2 Словарь данных

Детальное описание полей реляционной базы данных представлено в таблицах 9-17 для каждой из таблиц.

1. Справочник транспортных средств (vehicles)

Таблица 9 Справочник транспортных средств

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, значение автоинкрементируется, начиная с 1
name	varchar(100)	Наименование транспортного средства (отображаемое имя)
registration_number	varchar(20)	Государственный регистрационный знак
device_id	varchar(50) UNIQUE	Идентификатор бортового устройства, уникален для каждого ТС
type_id	int	Идентификатор типа транспортного средства (справочник)
department_id	int (FK)	Внешний ключ к таблице подразделений (departments)
created_at	timestamp	Дата и время создания записи

2. Справочник геозон (geozones)

Таблица 10 Справочник геозон

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, автоинкремент
name	varchar(100)	Название геозоны
type	varchar(10)	Тип геозоны: 'polygon' — многоугольник, 'circle' — окружность

Наименование	Тип данных	Значение
coordinates	jsonb	Координаты: для polygon — массив вершин, для circle — центр и радиус
is_active	boolean	Признак активности геозоны

3. Данные телеметрии (telemetry_packets)

Таблица 11 Данные телеметрии

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, автоинкремент
vehicle_id	bigint (FK)	Внешний ключ к таблице vehicles
packet_time	timestamp	Время из пакета EGTS, установленное бортовым устройством
reception_time	timestamp	Время приёма пакета сервером
latitude	double precision	Широта из пакета (позиционная подзапись или усреднённая)
longitude	double precision	Долгота из пакета
s2_cell	varchar(20)	Квадроключ S2 (идентификатор ячейки), вычисленный по координатам с уровнем, заданным в general_settings. Используется для пространственного поиска.

4. Дополнительные данные телеметрии (telemetry_subrecords)

Таблица 12 Дополнительные данные телеметрии

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, автоинкремент
packet_id	bigint (FK)	Внешний ключ к таблице telemetry_packets
record_type	smallint	Тип подзаписи EGTS (0 — позиционная, 1 — состояние и т.д.)
altitude	real	Высота над уровнем моря (м)
speed	real	Скорость (км/ч)
course	real	Направление движения (градусы)
odometer	real	Показания одометра (км)
sensors_data	jsonb	Данные дополнительных датчиков в структурированном виде
timestamp	timestamp	Время подзаписи (если отличается от времени пакета)

5. События телеметрии (telemetry_events)

Таблица 13 События телеметрии

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, автоинкремент
vehicle_id	bigint (FK)	Внешний ключ к таблице vehicles

Наименование	Тип данных	Значение
event_type	varchar(50)	Тип события (например, 'speed_exceed', 'harsh_braking')
event_time	timestamp	Момент возникновения события
value	real	Фактическое значение параметра, вызвавшего событие
threshold	real	Пороговое значение, заданное в настройках
subrecord_id	bigint (FK) опционально	Ссылка на конкретную подзапись, в которой зафиксировано событие
config_id	int (FK)	Внешний ключ к таблице telemetry_event_config

6. События геозон (geozone_events)

Таблица 14 События геозон

Наименование	Тип данных	Значение
id	bigint	Первичный ключ, автоинкремент
vehicle_id	bigint (FK)	Внешний ключ к таблице vehicles
geozone_id	bigint (FK)	Внешний ключ к таблице geozones
event_type	varchar(20)	Тип события: 'enter' — вход в зону, 'leave' — выход из зоны
event_time	timestamp	Время возникновения события

Наименование	Тип данных	Значение
subrecord_id	bigint (FK) опционально	Ссылка на подзапись с координатами, вызвавшую событие

7. Настройки событий телеметрии (telemetry_event_config)

Таблица 15 Настройки событий телеметрии

Наименование	Тип данных	Значение
id	int	Первичный ключ, автоинкремент
event_type	varchar(50)	Тип телеметрического события
parameters	jsonb	Параметры события (пороги, условия, длительность и т.д.)
is_enabled	boolean	Признак активности данного типа событий
description	text	Текстовое описание конфигурации

8. Настройки событий геозон (geozone_event_config)

Таблица 16 Настройки событий геозон

Наименование	Тип данных	Значение
id	int	Первичный ключ, автоинкремент
geozone_id	bigint (FK)	Внешний ключ к таблице geozones
event_type	varchar(20)	Тип события ('enter' или 'leave')

Наименование	Тип данных	Значение
is_enabled	boolean	Признак активности конфигурации

9. Общие настройки (general_settings)

Таблица 17 Общие настройки

Наименование	Тип данных	Значение
key	varchar(50) (PK)	Первичный ключ – строковый идентификатор настройки (например, 'geohash_level_db')
value	text	Значение настройки (может быть числом, строкой или JSON)
description	text	Описание назначения параметра

3.3 Описание разработки интерфейса пользователя

Интерфейс пользователя является графическим пользовательским интерфейсом, выполненным в веб-среде. Пользователь выполняет взаимодействие с приложением с помощью веб-браузера. Клиентское приложение является одностраничным приложением – загруженная страница обладает высоким уровнем интерактивности, выполняя запросы данных с помощью сценариев страницы, без загрузок новых страниц и перезагрузок текущей страницы.

В ходе использования системы отслеживания пользователи будут выполнять несколько групп задач.

3.3.1 Интерфейс ведения справочника объектов

Для определения перечня объектов мониторинга необходимо предоставить возможность создания, редактирования, обновления данных об этих объектах. В пользовательском интерфейсе этот функционал доступен в

меню «Устройства», в котором доступен просмотр существующих объектов мониторинга (рисунок 3) и их редактирование (рисунок 4).

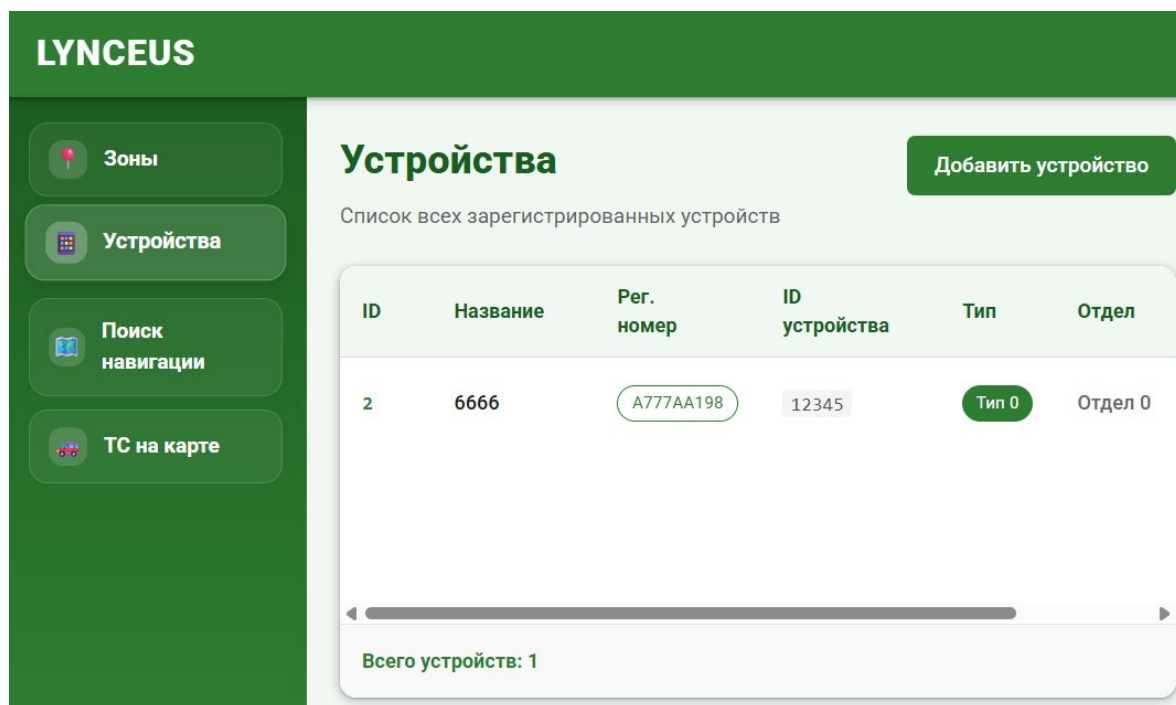


Рисунок 3 Интерфейс просмотра справочника

The screenshot shows a modal form titled 'Добавить новое устройство' (Add new device). It contains five input fields, each with a red asterisk indicating a required field:

- Название устройства ** (Device name): Input field with placeholder 'Введите название'.
- Регистрационный номер ** (Registration number): Input field with placeholder 'Введите регистрационный номер'.
- ID устройства (уникальный) ** (Device ID (unique)): Input field with placeholder 'Введите уникальный ID устройства'.
- Тип устройства (ID) ** (Device type (ID)): Input field with value '0'.
- Отдел (ID) ** (Department (ID)): Input field with value '0'.

At the bottom of the form are two buttons: 'Отмена' (Cancel) and 'Создать устройство' (Create device).

Рисунок 4 Интерфейс добавления и редактирования объекта мониторинга

Для настройки географических зон отслеживания пользователь должен воспользоваться меню «Зоны», где доступен просмотр зон на карте с метаданными (рис. 5) и их редактирование (рис. 6).

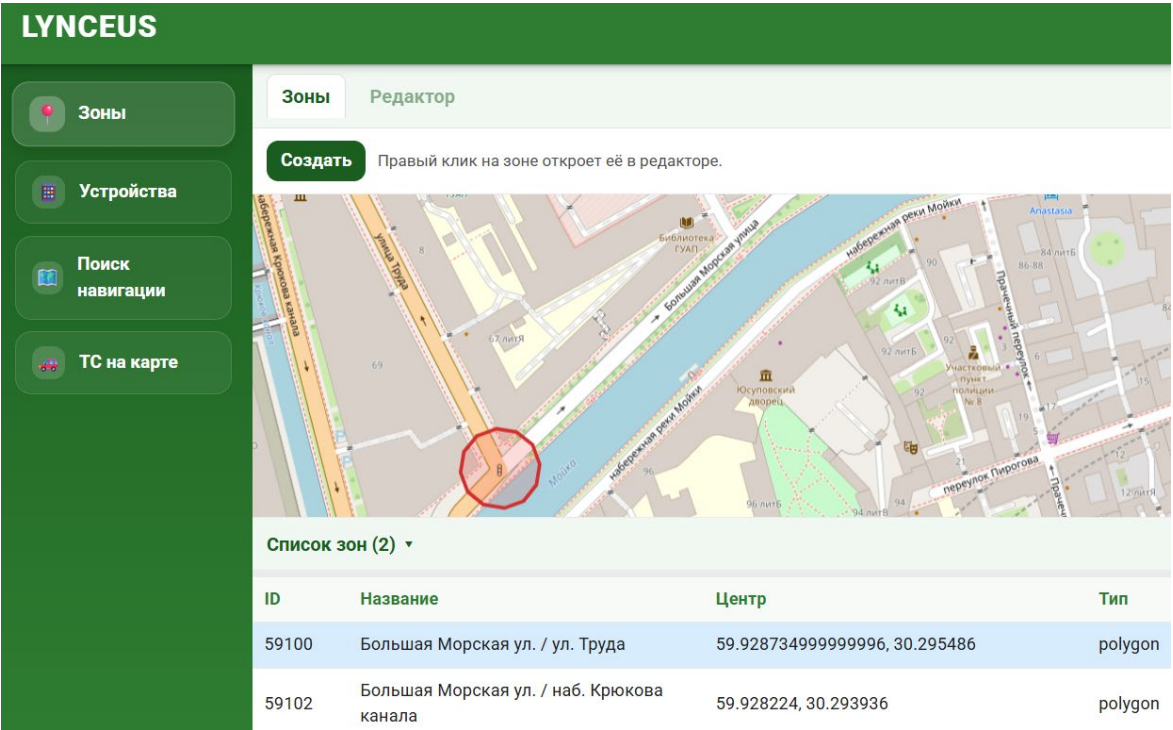


Рисунок 5 Просмотр геозон на карте

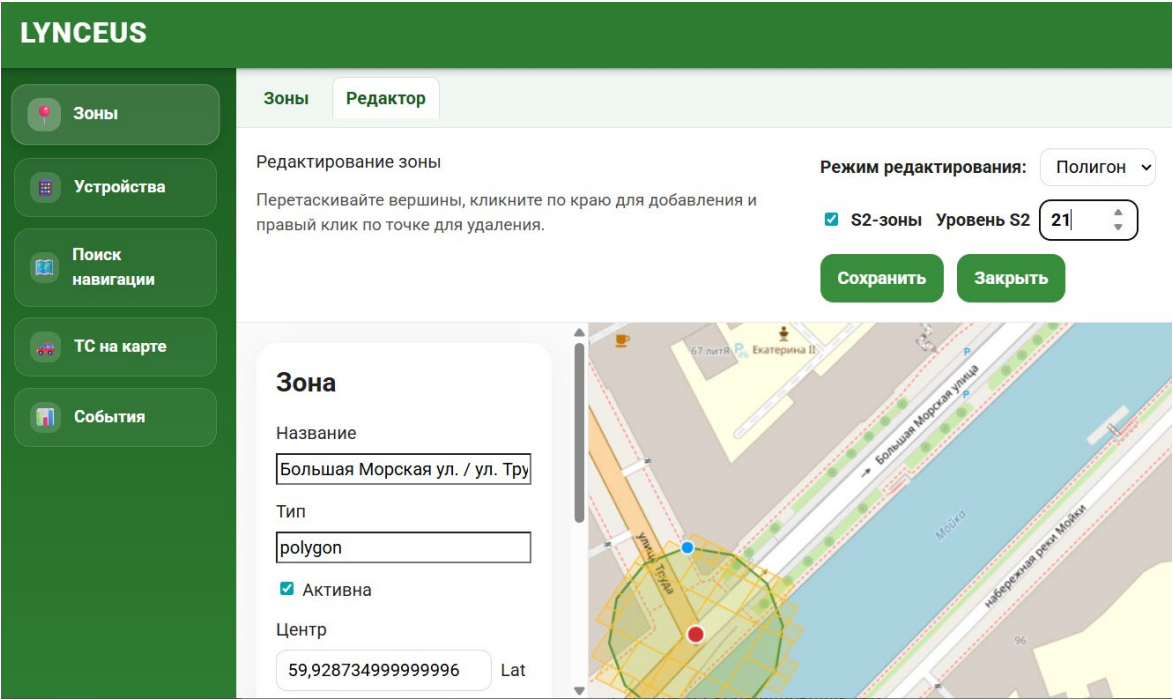


Рисунок 6 Редактирование геозон

3.3.2 Интерфейс мониторинга транспортных средств

Для определения наиболее актуального местоположения ТС пользователю предоставлена возможность просмотра карты, на которой метками отображена позиция и направление движения транспортных средств (рисунок 7). Положение устройств на карте динамически обновляется в видимом пользователю пространстве карты по мере поступления новых телеметрических данных. При наведении указателя мыши на маркер отображается всплывающее окно с телеметрическими данными.

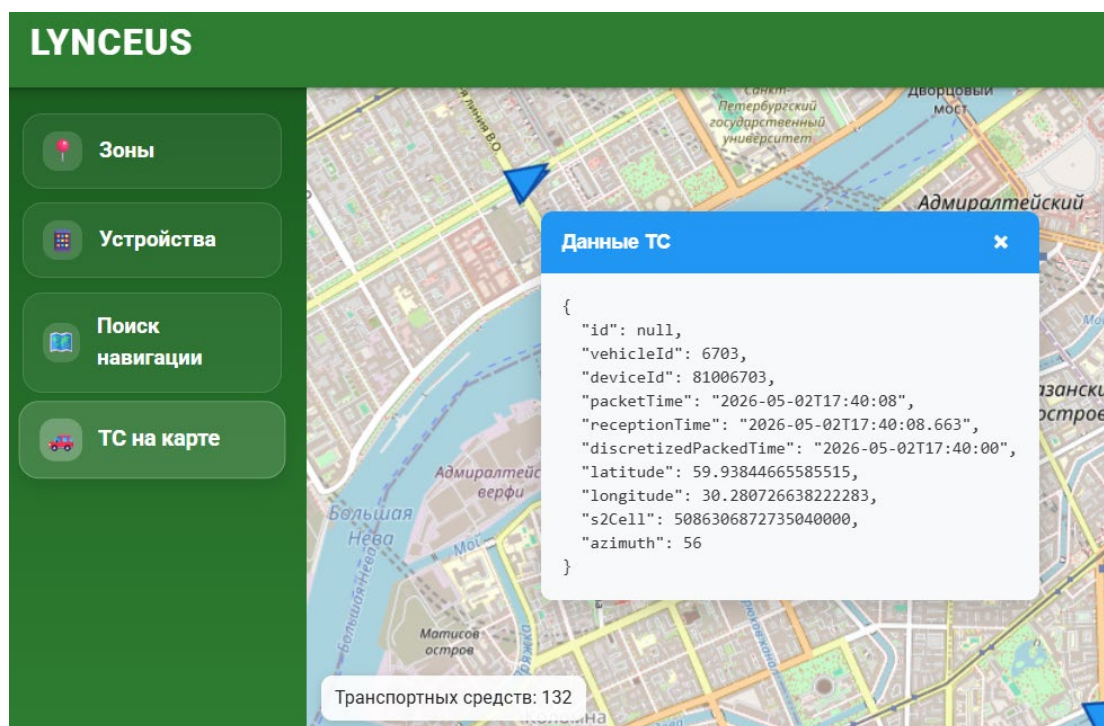


Рисунок 7 Актуальное местоположение на карте

Для просмотра исторических телеметрических данных может быть использовано меню «Поиск навигации». На вкладке «Навигация устройства» может быть получена история местоположения за период времени для указанного устройства (рис. 8).

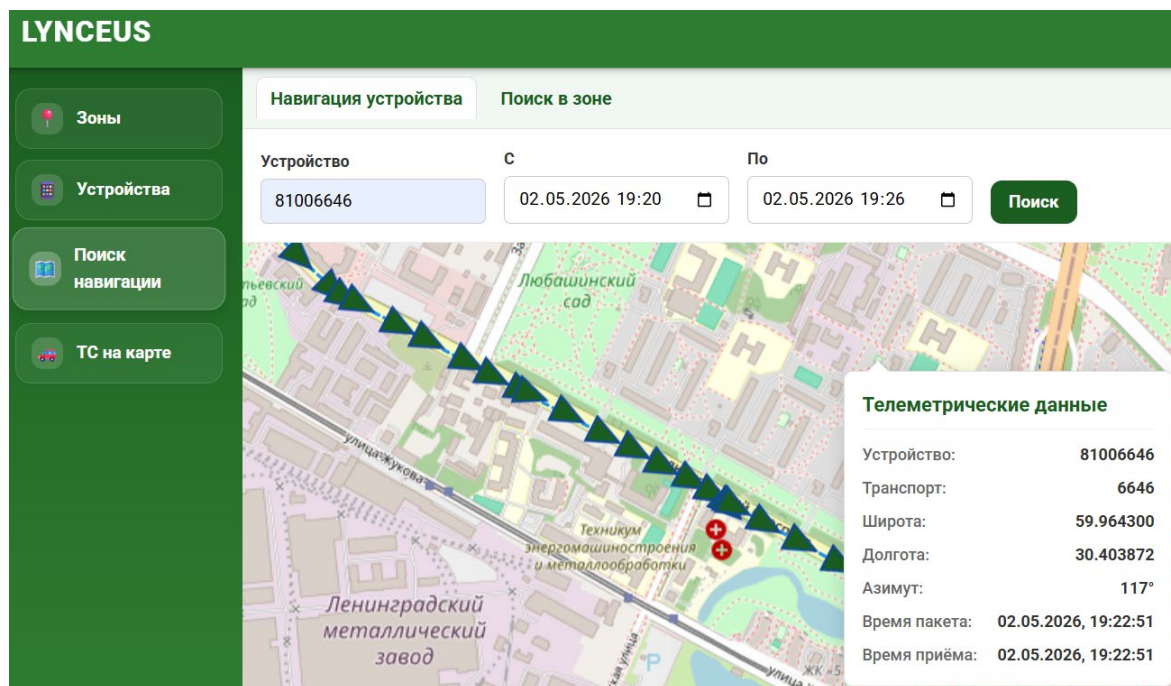


Рисунок 8 Исторические телеметрические данные

Одна из центральных функций системы – пространственно-временной поиск исторических телеметрических данных. Эта функция доступна на вкладке «Поиск в зоне» (рисунок 9). Пользователем указывается временной период и зона поиска (четырёхугольник), после чего выполняется поиск.

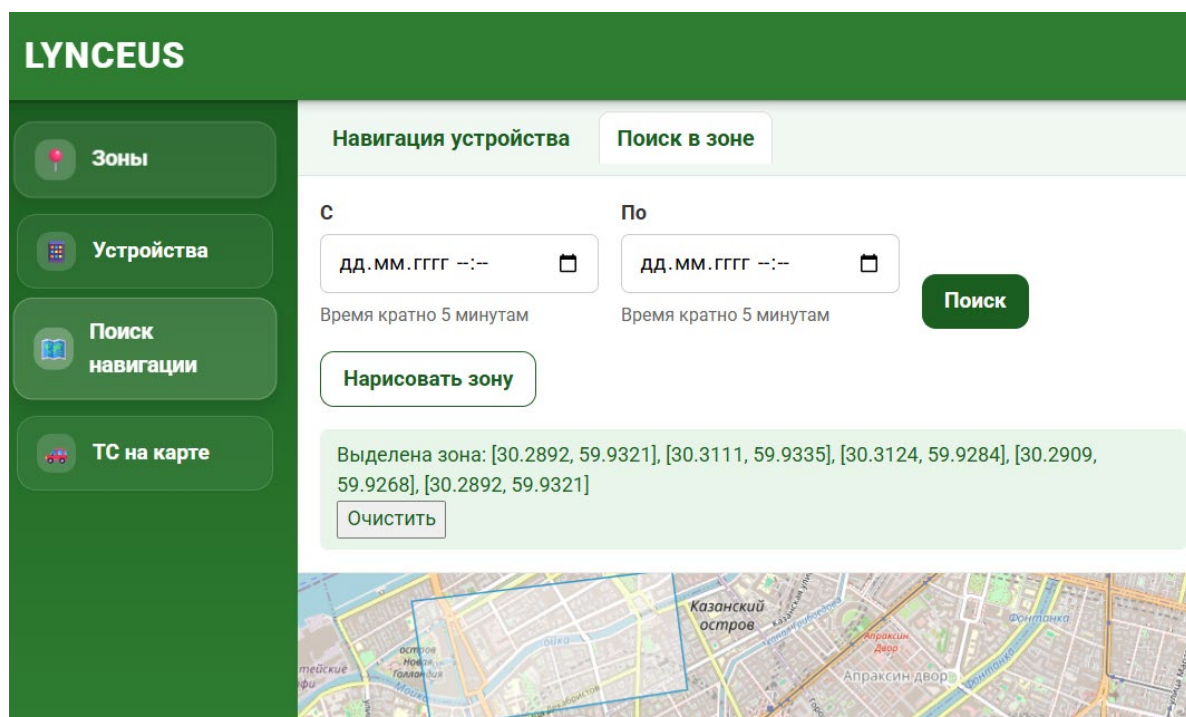


Рисунок 9 Пространственно-временной поиск

После выполнения поиска отображается список устройств с указанием временного периода, в котором устройство было в зоне (рисунок 10). С помощью кнопки «Построить трек» доступен быстрый запрос исторических навигационных данных на вкладке «Навигация устройства».

The screenshot displays the LYNCEUS web application interface. On the left is a dark green sidebar with navigation buttons: 'Зоны' (Zones), 'Устройства' (Devices), 'Поиск навигации' (Navigation Search), and 'ТС на карте' (Vehicles on map). The main content area has a light green header with two tabs: 'Навигация устройства' (Device Navigation) and 'Поиск в зоне' (Search in zone). Under the 'Поиск в зоне' tab, there are input fields for a time range from '02.05.2026 19:40' to '02.05.2026 20:40', both with calendar icons and a note 'Время кратно 5 минутам' (Time is a multiple of 5 minutes). There are 'Поиск' (Search) and 'Нарисовать зону' (Draw zone) buttons. Below this, a green box shows the selected zone coordinates: '[30.4351, 59.9664], [30.4340, 59.9601], [30.4470, 59.9602], [30.4474, 59.9672], [30.4351, 59.9664]' with an 'Очистить' (Clear) button. The bottom section, titled 'Найденные устройства в зоне' (Found devices in zone), lists two devices. Each entry includes the device ID, transport ID, and a time period, with a 'Построить трек' (Build track) button to the right.

Найденные устройства в зоне	
Устройство ID: 80006734 Транспорт ID: 6734 Период: 02.05.2026, 17:08:19 – 02.05.2026, 17:11:46	Построить трек
Устройство ID: 80006731 Транспорт ID: 6731 Период: 02.05.2026, 17:07:26 – 02.05.2026, 17:10:25	Построить трек

Рисунок 10 Результат пространственно-временного поиска

На основе созданных в справочнике геозон формируются события входа в зону и выхода из зоны для устройств. Пользователю доступен просмотр формируемых событий в меню «События». События показываются по мере поступления новых данных. Внешний вид интерфейса показан на рисунке 11.

LYNCEUS						
Зоны Устройства Поиск навигации ТС на карте События	События посещения зон					Подключить
	События в реальном времени (последние 500)					
	● Не подключено 500/500					
	Вход/Выход	ID устройства	ID ТС	ID зоны	Название зоны	Дата и время
	Вход	81006577	ТС 6577	Зона 63859	З КМ	03.05.2026, 12:22:51
	Выход	81006667	ТС 6667	Зона 65489	МОРСКАЯ НАБ.,19	03.05.2026, 12:22:51
	Вход	80006617	ТС 6617	Зона 51539	УЛ. БРАТЬЕВ РАДЧЕНКО, УГОЛ ПР. ЛЕНИ...	03.05.2026, 12:22:50
	Выход	81006715	ТС 6715	Зона 64452	ЛЕНСКАЯ УЛ., 18	03.05.2026, 12:22:50
	Вход	81006725	ТС 6725	Зона 62874	ГИПЕРМАРКЕТ "ЛЕНТА"	03.05.2026, 12:22:50

Рисунок 11 Отображение событий

ЗАКЛЮЧЕНИЕ

В ходе выполнения производственной практики выполнены этапы проектирования и реализации программного продукта – системы мониторинга транспортных средств с обработкой событий в реальном времени.

Проведен анализ предметной области: определены требования к продукту, выполнено сравнение с аналогами, показана целесообразность разработки программного продукта.

Выполнено проектирование продукта: определены технические и программные средства разработки, тестирования и развертывания ПО.

На этапе реализации описана архитектура системы, приведено описание каждой из подсистем, их назначение, формат обмена данными и алгоритмы их работы. Указано описание структур данных. Описан в текстовом и графическом виде пользовательский графический интерфейс.

Задачи работы выполнены, цель достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Росстат - Транспорт // Росстат URL: <https://rosstat.gov.ru/statistics/transport> (дата обращения: 03.05.2026).
2. Постановление Правительства Российской Федерации "Об утверждении Правил оснащения транспортных средств категорий М2, М3 и транспортных средств категории N, используемых для перевозки опасных грузов, аппаратурой спутниковой навигации" от 22.12.2020 № 2216 // Официальное опубликование правовых актов. 2020 г. № 0001202012250015.
3. Постановление Правительства Российской Федерации "О взимании платы в счет возмещения вреда, причиняемого автомобильным дорогам общего пользования федерального значения транспортными средствами, имеющими разрешенную максимальную массу свыше 12 тонн" от 14.06.2013 № 504 // Официальное опубликование правовых актов. 2013 г. № 0001201306180005.
4. Зиарманд Артур Нисарович, Хаханов Владимир Иванович. Модели и методы мониторинга и управления транспортом // Радиоэлектроника и информатика. 2016. №3.
5. Игумнов Артем Олегович АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ РАСПРЕДЕЛЕННОЙ СИСТЕМЫ МОНИТОРИНГА И УПРАВЛЕНИЯ ТРАНСПОРТОМ // Доклады ТУСУР. 2021. №2.
6. Кушнир Надежда Владимировна, Вонарх Юлия Сергеевна, Деев Илья Максимович, Долунц Анастасия Станиславовна, Карпенко Никита Андреевич АНАЛИЗ ДАННЫХ В РЕАЛЬНОМ ВРЕМЕНИ: ПРЕИМУЩЕСТВА И ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ STREAM PROCESSING // Научный журнал КубГАУ. 2025. №207.
7. Баженов Артём Эдуардович, Райков Александр Вячеславович, Нехаев Максим Вадимович, Ореховский Никита Викторович. Оптимизация взаимодействия микросервисов с использованием асинхронных вызовов и

брокеров сообщений (Kafka, RabbitMQ) // Программные системы и вычислительные методы. 2025. №4.

8. Тюкачев Николай Аркадиевич Алгоритм определения принадлежности точки многоугольнику общего вида или многограннику с треугольными гранями // Вестник ТГТУ. 2009. №3.

9. Долгий П. С. ПРОГРАММНЫЕ ОСОБЕННОСТИ РАЗРАБОТКИ ГИС-ПРОЕКТА «ГЕОДИНАМИКА И ТЕХНОГЕНЕЗ БЕЛАРУСИ» // Вестник Полоцкого государственного университета. Серия F. Строительство. Прикладные науки. 2023. №1.

10. Darius Šidlauskas, Simonas Šaltenis, Christian W. Christiansen, Jan M. Johansen, and Donatas Šaulys. Trees or grids? indexing moving objects in main memory. // In Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS '09). Association for Computing Machinery, New York, NY, USA, 236–245. 2009.

11. Geospatial Grid System // Microsoft Learn URL: <https://learn.microsoft.com/ru-ru/kusto/query/geospatial-grid-systems?view=azure-data-explorer> (дата обращения: 03.05.2026).

12. Al-Nsour, Esam & Sleit, Azzam & Alshraideh, Mohammad. A Pictorial Performance Comparison of Spatial Indexes. // 11th International Conference on Information and Communication Systems (ICICS) 042-047. 2020.

13. Толмачев Павел Владимирович PostgreSQL 16. Оптимизация запросов // Москва, Постгрес Профессиональный, 2026.

14. П. И. Смирнов, М. Ю. Карелина, Б. С. Субботин. Применение систем транспортной телематики для повышения эффективности эксплуатации техники // Министерство науки и высшего образования Российской Федерации, Вологодский государственный университет. – Вологда: ВоГУ, 2023. с. 123-124

15. JIANG Bohui, ZHOU Weifeng. Comparative Analysis of GeoHash, Google S2 and Uber H3 as Global Geographic Grid Coding Methods // Geography & Geographic Information Science. 2024. №40.
16. Marcin Gorawski, Adam Dyga. Indexing of spatio-temporal telemetric data based on distributed mobile bucket index // Proceedings of the 24th IASTED international conference on Parallel and distributed computing and networks. 2006. С. 292-297.
17. ГОСТ 33465-2015. Глобальная навигационная спутниковая система. Система экстренного реагирования при авариях. Протокол обмена данными устройства/системы вызова экстренных оперативных служб с инфраструктурой системы экстренного реагирования при авариях // Москва: Стандартинформ, 2017. 65 с.
18. Performance Impact of 10,000+ Geofences in Traccar // Traccar URL: <https://www.traccar.org/forums/topic/performance-impact-of-10000-geofences-in-traccar/> (дата обращения: 03.05.2026).
19. Ограничения // Wialon Справочный центр URL: <https://help.wialon.com/ru/wialon-local/2304/user-guide/monitoring-system/limitations> (дата обращения: 03.05.2026).
20. Системные требования // Fort Monitor URL: <https://support.fort-monitor.ru/article/30936> (дата обращения: 03.05.2026).

Приложение А

Минимальные системные требования для развертывания всех модулей системы при нагрузке, не превышающей 2000 сообщений в секунду:

- Процессор (CPU): Intel Core i7 (3-го поколения) или аналог, частота от 3.4 ГГц (4 ядра / 8 потоков);
- Оперативная память (RAM): от 8 ГБ (DDR3 и выше);
- Диск: SSD от 100 ГБ свободного места;
- Система: Linux (Ubuntu 22.04 LTS / Debian 11 / другая с поддержкой Docker) или Windows 10/11 с WSL2;
- Интернет: от 100 Мбит/с (для загрузки образов и обновлений).